# *preface*

I was in university when the World Wide Web first came to be, and I remember using the Mosaic browser in its early form. When I entered grad school, we were already using the web to write content for students, including both static HTML and early CGI. I graduated from single Perl scripts to working with Java, eventually working with large teams of developers, writing scaled systems for the banking and telecom sectors.

In 2008, I was managing development teams at a company that got acquired, and during the transition, I had a lot of time on my hands. I decided to pick up a new language: Python. I used it to solve a couple of smaller problems we had at work and pretty much have never looked back. My next company was a start-up, building web tools. I looked around in the Python ecosystem for a web framework and landed on Django. Even back in its 1.1 days, Django was a complete system. It included everything we needed to build websites, which allowed my team and me to concentrate on the code specific to our clients.

Django's core is based on mapping a URL to some code, running the code that generates a page and returning that page as a response. The page can be built using Django's template engine, meaning HTML can be composed like code, removing the need for repetition. Most websites require some storage mechanism, and Django's object relational mapping (ORM) abstracts away the database, making it easier to query and manipulate data in Python.

This alone would be enough to recommend Django to build websites, but it contains much more. Out of the box, Django is built for multiuser websites, meaning it includes mechanisms for authentication, authorization, and user management. It comes with tools to help write automated tests, both at the unit level and with hooks for tools like Selenium for doing cross-site integration testing.